

Docker

1) Virtualisation par conteneurs : l'idée clé

Conteneur vs machine virtuelle (VM)

- **VM** : virtualise du matériel (OS complet par VM) → plus lourd (RAM/CPU), démarrage plus lent.
- **Conteneur** : virtualise au niveau **OS** (partage du noyau Linux) → plus léger, plus rapide, plus dense (plus d'instances sur la même machine).

Pourquoi Docker est “possible” sur Linux ?

Docker s'appuie principalement sur deux briques du noyau Linux :

- **Namespaces** : isolation (ce que le conteneur “voit”).
- **Cgroups** : limitation/gestion des ressources (ce que le conteneur “consomme”).

2) Namespaces : isolation

Rôles principaux

1. **Isolation des ressources** : chaque conteneur “croit” être seul.
2. **Sécurité / cloisonnement** : un conteneur ne voit pas les processus/ressources des autres (sauf partage explicite).
3. **Flexibilité** : combinaison de plusieurs namespaces pour un environnement cohérent (proche d'une VM, mais beaucoup plus léger).

Types importants (à connaître)

- **PID namespace** : isole les processus (un ps dans le conteneur ne montre que “son” arbre).
- **NET namespace** : isole la pile réseau (interfaces, routes, ports, IP).

- **MNT namespace** : isole les montages / FS (hiérarchie de répertoires propre).
- **USER namespace** : isole utilisateurs/groupes (root “dans” le conteneur peut être mappé sur un user non privilégié côté hôte).

3) Cgroups : limitation, partage équitable, observabilité

À quoi ça sert ?

1. **Limitier** CPU, mémoire, disque, réseau (évite qu’un conteneur “buggué” prenne toute la RAM).
2. **Partager équitablement** : quotas / poids / priorités entre conteneurs.
3. **Surveiller** l’usage (métriques exploitées par docker stats et par les orchestrateurs).

Résumé mental

- **Namespaces** = “je ne vois que mon monde”
- **Cgroups** = “je ne dépasse pas mon budget ressources”

4) Architecture Docker : client/serveur

Docker fonctionne en **client/serveur** :

- Le **client** (CLI docker ...) envoie des commandes
- Le **daemon** (moteur Docker) exécute : images, conteneurs, réseaux, volumes...

5) Images vs conteneurs

Définitions

- **Image** : “template” immuable (lecture seule), composée de **couches (layers)**.

- **Conteneur** : instance en exécution d'une image + **couche writable** au-dessus.

OverlayFS (important)

- Empile des couches (layers) pour donner une **vue unifiée**.
- Optimise le disque : une même couche (ex : ubuntu:20.04) n'est stockée qu'une fois même si utilisée par 10 conteneurs.
- Isolation : chaque conteneur a sa couche supérieure en écriture indépendante.

Manifest

Le **manifest** est la "carte d'identité" de l'image :

- décrit couches + configuration
- garantit l'intégrité (hash SHA256)
- permet d'éviter les téléchargements redondants (Docker sait quoi récupérer et assembler).

6) Registres, dépôts, tags

Où sont stockées les images ?

- **Registry** (registre) : contient des dépôts
- **Repository** (dépôt) : contient plusieurs versions d'une image
- **Tag** : version (ex : nginx:latest, redis:6.0.8)

Officiel vs non officiel

- Image officielle : qualité, sécurité, doc, best practices attendues.
- Non officiel : ex bitnami/redis:6.0.8.

7) Commandes images : l'essentiel

- Télécharger : `docker pull image:tag`
- Lister images locales : (diapo "afficher le contenu du dépôt local") → typiquement `docker images`
- Inspecter une image : `docker inspect nginx:latest`
- Supprimer : `docker rmi <IMAGE_ID>`

Exercice vu : pull Memcached + Nginx, vérifier présence, supprimer, revérifier.

8) Conteneurs : création, cycle de vie, commandes

docker run

(la commande pivot)

Syntaxe :

`docker run [OPTIONS] IMAGE:TAG [COMMAND] [ARG...]`

Effet : **crée + démarre** un conteneur en une seule opération.

Options à connaître

- `-it` : mode interactif + pseudo-terminal
- `--name` : nom du conteneur
- `--rm` : supprime automatiquement le conteneur à l'arrêt
- `-d` : mode détaché (daemon / arrière-plan)

Commandes de gestion

- `docker ps` : conteneurs en cours d'exécution
- `docker ps -a` : tous (y compris arrêtés)
- `docker inspect <container>` : infos détaillées conteneur

Exemples typiques

- Lancer un conteneur très léger :
 - `docker run alpine:latest sleep 5`
 - `docker run --rm -it alpine sh`
- Nginx :
 - bloquant : `docker run nginx`
 - non bloquant : `docker run -d nginx`

Entrer dans un conteneur en cours

- `docker exec -it <nomConteneur> /bin/bash`
(utile pour aller voir `/usr/share/nginx/html` côté nginx).

9) Réseau et ports : “exposer” vs “mapper”

Point clé : un service peut écouter sur un **port interne** dans le conteneur, mais tu dois décider comment l'atteindre depuis l'hôte.

Mapping automatique

- `-P` : mappe automatiquement **tous les ports exposés** du conteneur vers des ports hôtes aléatoires.

Ex : `docker run -P -d nginx`

Mapping manuel

- `-p` : définit le mapping exact
 - `-p 80` : port conteneur 80 mappé sur un port hôte choisi automatiquement
 - `-p 8000:80` : port hôte 8000 → port conteneur 80

Ex : `docker run -d --name webServer2 -p 8000:80 nginx`

10) Volumes : persistance des données

Pourquoi ?

Sans volume :

- la couche writable du conteneur est **volatile**
- si tu supprimes le conteneur, tu perds les données

Les **volumes** permettent de **persister** et/ou **partager** des fichiers.

Bind mount (vu en exemple)

- `-v <dossier_hôte>:<dossier_conteneur>`

Ex nginx :

```
docker run -d -P -v /Users/.../WebDocker:/usr/share/nginx/html nginx
```

11) Variables d'environnement (ex : DB)

Exemple MySQL :

```
docker run -p 3306:3306 --name mysqlDB -e MYSQL_ROOT_PASSWORD=root -d mysql:latest
```

À retenir :

- `-e` injecte une variable d'environnement
- beaucoup d'images (DB, caches, apps) se configurent via env vars

12) Construire des images

Le cours montre deux voies :

A) Créer une image depuis un conteneur (approche “snapshot”)

Étapes vues :

1. créer une image à partir d'un conteneur
2. se logger sur Docker Hub
3. publier l'image sur Docker Hub

(Pratique pour démo, moins recommandé en production car moins "reproductible".)

B) Dockerfile (approche standard / industrielle)

- Construire une image à partir d'un **Dockerfile**
- Notion d'image "déclarative" et reproductible

Point d'attention mentionné :

- installer des paquets sans mise à jour des dépendances peut poser problème (sécurité/versions).

Exercice Flask

- conteneuriser une petite app web Python (Flask)
- nom d'image demandé : app_web:1.0

13) Déploiement (notions du cours)

- Service cloud mentionné : Render (render.com)
- Déploiement possible à partir de GitHub (pipeline simple).

14) YAML : prérequis pour Docker Compose

Ce qu'il faut savoir

- YAML = format de représentation de données (pas un langage de balisage)
- indentation (type Python), **tabulations interdites**, espaces uniquement
- extensions : .yaml / .yml
- structures : **mapping (dictionnaire)** et **séquences (listes)**

Types de données :

- scalaires : string, nombre, date, booléen
- multi-doc, multi-lignes, imbrications (séquences/mappings)

15) Docker Compose : multi-conteneurs

Définition

Docker Compose simplifie le déploiement et la gestion d'applications **multi-conteneurs** via un fichier **YAML** (souvent docker-compose.yml).

Utilités clés

- définir plusieurs services (API, DB, reverse proxy, cache...)
- tout lancer avec une commande (docker compose up)
- gérer dépendances entre services
- portabilité (même config pour toute l'équipe)
- réseau simplifié : communication par **nom de service** (ex db:3306)

Exemple “web + redis”

Le cours décrit un scénario où :

- web-fe sert une page et incrémente un compteur dans redis
- réseau commun counter-net
- volume counter-vol monté pour persister redis

Points YAML mis en évidence :

- services: : définition des conteneurs (nom = nom de service)
- build: : construire depuis Dockerfile local
- command: : override commande de lancement
- ports: : hôte:conteneur
- networks: : réseau(s) attaché(s)
- volumes: : volumes déclarés / montés (type volume, source, target)

16) Commandes Docker Compose (à mémoriser)

- docker compose up : crée images, conteneurs, réseaux, volumes nécessaires
 - souvent avec --detach (arrière-plan)
 - -f pour un fichier compose non standard
- docker compose stop : stop sans supprimer
- docker compose restart : redémarre
- docker compose ps : liste les conteneurs et leur état
- docker compose down : stop + supprime conteneurs et réseaux (par défaut **pas** volumes/images)
- nettoyage complet :
 - docker-compose down --volumes --rmi all

17) Exercice final : Load Balancing avec Nginx + Compose

Objectif : 2 serveurs web derrière un reverse proxy Nginx qui répartit la charge.

Exigences :

- web1 et web2 :
 - répondent sur port interne 80
 - messages différents (“Serveur 1” / “Serveur 2”)
- nginx :
 - reverse proxy + load balancing
 - fichier nginx.conf avec upstream listant web1 et web2
 - écoute sur **8080** côté hôte
- réseau personnalisé : app-net (tous les services dessus)
- test : http://localhost:8080, refresh → alternance des réponses

Mini “cheat sheet” (résumé opérationnel)

- **Image** : pull / images / inspect / rmi
- **Conteneur** : run / ps / ps -a / inspect / exec
- **Ports** : -P auto, -p hôte:conteneur manuel
- **Persistence** : volumes -v (bind mount) ou volumes gérés Docker (Compose)
- **Config runtime** : -e VAR=...
- **Multi-services** : Compose (services/networks/volumes + docker compose up/down/ps)